Open Access | https://doi.org/10.25080/gaxm4526



SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Solving the AI Eval Gap: Domain-Aware Evaluation for Production AI Agents

Shivika Bisen[1]

[1]Independent

Abstract

AI agents are moving into enterprise production faster than the tooling used to evaluate them. Generic evaluation frameworks measure hallucination rate, latency, and token cost, but not the question that actually blocks a launch: did the agent do the *right thing* for this schema, these documents, this domain? Today every team hand-labels golden sets that go stale the moment data, prompts, or use cases change, producing non-reproducible pipelines that do not transfer across teams.

We present **AI Eval Engine**, an open-source Python framework that treats evaluation as a pipeline to be *generated* rather than an artifact to be *authored*. A team points the framework at where its domain lives; the framework (1) ingests a pluggable domain context from a lightweight configuration, (2) builds a living **Domain Compliance Runbook** of domain facts and compliance criteria, (3) generates a versioned, domain-grounded golden set from those criteria, (4) emits a runnable evaluation script that scores correctness, evidence grounding, and output format — feeding failures back into the runbook, and (5) tracks the result as quarter-over-quarter objectives and key results. A central methodological concern for any pipeline that uses a language model both to generate tests and to judge them is *self-bias*; we address it by anchoring demonstrations on tasks whose correctness can be verified by execution or by grounding in source documents, and where the model demonstrably fails without the ingested domain context. We illustrate the framework on two contrasting agent shapes — a structured, execution-scored scientific-coding agent (ScienceAgentBench) and an open-ended, grounding-scored financial document question-answering agent (FinanceBench) — showing that one reusable pipeline spans both without per-domain scratch tooling.

Keywords AI agents, evaluation, large language models, domain-aware evaluation, golden set generation, LLM-as-judge, safety, retrieval-augmented generation

Published Jul 13, 2026

Correspondence to
Shivika Bisen
shivikakbisen@gmail.com

Open Access

Copyright © 2026 Bisen. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

AI agents are moving into enterprise production faster than the tooling used to evaluate them. Teams shipping agents over financial filings, scientific datasets, or support transcripts hit the same wall: existing tooling measures generic LLM properties — hallucination rate, latency, token cost — but not the question that blocks a launch: *did the agent do the right thing in this domain?*

Domain correctness is not a generic property. A data agent over a financial schema and a RAG agent over scientific documents require different notions of "correct," and those notions exist only in the **domain context**: a scientific agent must load the right columns and compute the metric the paper reported; a financial agent must ground every figure in its filing and decline when the filing does not support an answer. Neither constraint is visible to a generic judge scoring token-level fluency.

The state of practice is the **manually labeled golden set**, with three well-known failure modes: it goes *stale* when data or prompts change; it is *non-reproducible* across teams; and it gives *no clarity on what to fix*, reporting only that something broke. These follow from treating evaluation as an artifact to be *authored* rather than a pipeline to be *generated*.

This paper presents **AI Eval Engine**, an open-source Python framework that reframes evaluation as a domain-driven generator. The team points at where the domain lives; the framework produces a versioned golden set, a runnable evaluation script, and a living Domain Compliance Runbook that captures emergent failure modes — shifting the human role from *author* to *reviewer* and letting teams track agent behavior as a first-class quarter-over-quarter OKR. It leans on a language model — Anthropic's Claude [1], via the Claude API and Claude Code — for context extraction, golden-set and eval-script generation, and the LLM-as-judge step. Using one model to both *generate* and *judge* tests raises a real objection — self-bias — which we confront in Methodology: Domain-Aware Evaluation Without Self-Bias and which drives our dataset choice.

2. Background and Related Work

Open-source evaluation frameworks tackle adjacent slices. **RAGAS** scores RAG systems on faithfulness, answer relevancy, and context recall [2]; **ARES** trains lightweight RAG judges [3]; **G-Eval** formalizes LLM-as-judge with chain-of-thought scoring [4], a practice characterized by L. Zheng *et al.* [5]; HELM standardizes *which* metrics are reported [6]; **DeepEval** [7] is a pytest-style harness, and **Arize Phoenix** [8] and **Comet Opik** [9] focus on production observability. In every case the *test cases* are assumed to exist. The closest neighbors instead generate them: G. Guinet, B. Omidvar-Tehrani, A. Deoras, and L. Callot [10] build task-specific exams from a corpus, EvalGen aligns LLM evaluators with human preferences [11], and SPADE synthesizes data-quality assertions [12]; agent-centric systems — the Agent-Testing Agent [13], TestAgent [14], and τ-bench [15] — push toward domain-specific benchmarking, with surveys [16], [17] and safety benchmarks TRIDENT [18] and DecodingTrust [19] mapping the landscape.

AI Eval Engine differs in two ways: it treats **automated generation of the test set itself**, driven by a pluggable domain context, as the primary contribution; and it accumulates a **living Domain Compliance Runbook** of domain constraints from observed failures, closing a loop back into generation. It is meant to be used *alongside* these tools — RAGAS or DeepEval metrics register as extra Step-4 scorers — and self-improving methods such as Agentic Context Engineering [20] are candidates for evaluation *by* it, not components of it.

3. Methodology: Domain-Aware Evaluation Without Self-Bias

A pipeline that uses an LLM to *generate* the golden set and the same model family to *judge* the agent invites an obvious objection: the evaluation may simply reward outputs that look like what the generator would produce. This **self-bias** is the central risk of LLM-as-benchmark-generator plus LLM-as-judge pipelines [21], and the first thing a reviewer will probe.

We adopt a single guiding principle that defuses self-bias, benchmark saturation, and training contamination at once: **choose tasks where the model fails without the ingested domain context, and where correctness can be checked independently of the judge.** Two consequences follow.

- **Prefer verifiable correctness over judge opinion.** Where output can be *executed* (does the program produce the expected artifact?) or *grounded* (is the answer supported by cited evidence? — a claim of "capital expenditure was $1,577 million"

either matches the cited cash-flow statement or it does not), correctness is decided by the world, not the model.
- **Prefer domains the model could not have memorized.** Generic code generation (HumanEval-style) is saturated and largely in pretraining, so a high score measures recall, not domain-aware evaluation; the contribution is meaningful only on tasks needing context — a specific dataset or filing — the model never saw.

This principle makes the datasets in Demonstration Use Cases load-bearing: each was chosen because the agent cannot succeed without the ingested domain context, and each admits an execution- or grounding-based check independent of the judge. One scoping note: this principle governs Step 4 — scoring an agent's *answers*. Results does not run Step 4; it evaluates Step 3 — the *questions* the generator produces — with rule-based, offline metrics only, so no number there rests on a model grading its own output.

4. The Five-Step Framework

The framework decomposes evaluation into five steps (Figure 1), each an independent Python module with a stable interface so teams can replace, extend, or skip a step without forking the pipeline.

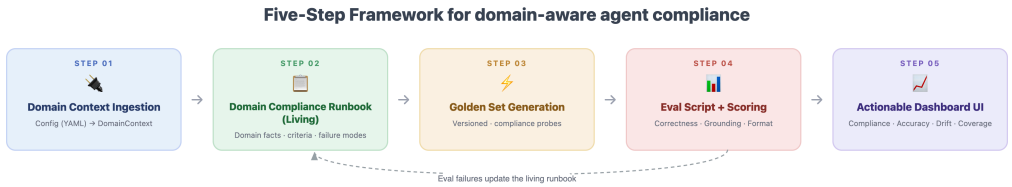


Figure 1. *The five-step framework. A pluggable domain context (Step 1) builds the living* ***Domain Compliance Runbook*** *(Step 2), whose criteria drive golden-set generation (Step 3); eval-script generation and scoring (Step 4) feed failures back into the runbook, and an actionable dashboard (Step 5) tracks the OKRs over time.*

4.1. Step 1 — Pluggable Domain Context Ingestion

The user provides a lightweight YAML configuration that points to one or more domain sources:

```yaml
project: financebench-qa-agent
domain_sources:
  - type: csv
    path: data/financebench/financebench.csv
    description: open-book QA over corporate 10-K filings
stratify_by: company
sample_per_stratum: 2
task:                    # how each row becomes an eval task
  kind: grounded_qa
  input_field: question
  gold_field: answer
  grounding_field: evidence
  category_field: company
```

The framework reads a representative, stratified sample from each source and issues a Claude API call that extracts the *implicit* domain structure into a typed `DomainContext` object — not a prose summary. The extraction captures the categories and taxonomies present in the data, the safety and compliance constraints the domain implies (e.g., "every stated figure must trace to a cited passage"), and quality signals (distributions, ambiguity clusters, missing-value patterns). Users are **not** asked to hand-author a taxonomy or compliance document; they point to where the data lives, and the framework surfaces what matters.

Stratified sampling ensures the extraction sees the domain's breadth, not only its most common cases.

4.2. Step 2 — Domain Compliance Runbook (Living)

From the `DomainContext`, the framework assembles a single content-addressed `domain_compliance_runbook.json` — the **Domain Compliance Runbook**. The name borrows from operations, where a *runbook* is the reference document an on-call engineer follows during an incident; here it is the domain reference the golden-set generator consults on every run. It has three sections: **domain facts** (definitions, conventions, units), **compliance criteria** (five *fixed, universal* agentic-compliance anchors — domain scope, evidence grounding, privacy/confidentiality, no biased advice, human escalation — instantiated per domain with severity: finance's MNPI rule — undisclosed information that could move a stock price, which securities law bars trading on or selectively revealing — is the privacy anchor's finance instance), and **common failure modes**. The first two are seeded now, at ingestion; the third is *living* — it accumulates as the eval (Step 4) surfaces failures, which the framework clusters deterministically by `(category, failure_type)` and turns into a `recommended_check` (the golden-set addition that would catch each one). The anchor taxonomy is therefore *declared* — fixed by design, so all five rules are tested in every domain — while its instantiations and failure modes are *discovered*: a cluster surfaced in run N becomes a runbook entry the golden-set generator (Step 3) picks up as a priority for run $N+1$, closing the loop between observation and test generation. A concern fitting none of the five is the signal to promote a new universal anchor, not to bolt on a one-off rule. The clustering and `recommended_check` mapping are **deterministic rules, not a learned model** — "living" means an accumulated, human-reviewable record, not gradient training.

4.3. Step 3 — Automated Golden Set Generation

From the Domain Compliance Runbook's criteria, the framework constructs a versioned, domain-grounded set of `GoldenCase`s, each carrying the input, the expected outcome (an executable check, an artifact, or an evidence-grounded answer), and the compliance criterion or capability it tests. Two paths exist: **normalize** (`build_golden_set`, fully offline) adopts a public benchmark's own verified labels; **generative** (`generate_golden_set`) has Claude author fresh cases grounded in real evidence — happy-path lookups, hard multi-step computations, definitionally ambiguous queries, and out-of-policy requests the agent must decline (e.g. asking for free cash flow from a balance sheet alone). In both, **guardrail tests** — cases whose correct outcome is a refusal or a hand-off to a human — are synthesized from the runbook's five anchors, so every domain rule is always tested. Generating this *compliance coverage* from extracted constraints, rather than plain corpus-to-question pairs, is the contribution; recurring failures recorded in the runbook (Step 2) feed back here as priorities for the next set.

Both modes enforce one rule: every figure in an expected answer must be traceable to cited evidence, or the case becomes an explicit refusal — the self-bias guard from Methodology: Domain-Aware Evaluation Without Self-Bias, letting a reviewer (or, where a public split exists, the benchmark's own gold answer) verify each case without trusting the generator. The FinanceBench result in Results uses **generative** mode — the framework authors its own golden set, which is exactly what we evaluate there. Golden sets are content-addressed (`goldensets/<version>/`) so reruns can be diffed, and a human reviewer may accept, edit, or reject any case before it is promoted — the human role is **reviewer**, not author.

4.4. Step 4 — Eval Script Generation and Scoring

The framework emits a runnable Python evaluation script producing three orthogonal scores per case: **correctness** (execution of the produced program for executable cases, or

a normalized / numeric match for grounded ones), **grounding** (the fraction of the answer supported by the cited evidence, by deterministic token/numeric overlap, so a fluent but unsupported figure is penalized), and **format validation** (structural integrity, checked without a model call). All three scorers in Results are **deterministic** — no model grades another model's output. Scorers are pluggable: the shipped `grounded_qa` and `code_execution` cover the two demos, and metrics from tools such as RAGAS or DeepEval can register as additional scorers. The LLM-as-judge is the designed fallback for the genuinely open-ended residue but was not exercised here: the judge is never the sole arbiter on a task that execution or grounding can settle. Failures from this step feed back into the Domain Compliance Runbook (Step 2).

4.5. Step 5 — Actionable Dashboard UI

Once the pipeline has run, the framework becomes a live OKR tracker: as data changes, golden sets and the Runbook regenerate on rerun, enabling quarter-over-quarter tracking of an **Agent Compliance Score** (pass rate on compliance-critical cases), a **Domain Accuracy Score** (correctness on in-domain queries), a **Drift Indicator** (share of new failure patterns since the last run), and a **Coverage Score** (share of the discovered domain context represented in the golden set). These four metrics are computed from Step-4 runs and are defined here but not measured in this paper.

5. Demonstration Use Cases

We demonstrate the framework on two contrasting, domain-dependent agent shapes that bracket the spectrum it targets: a **structured, execution-scored** scientific coder (ScienceAgentBench) and an **open-ended, grounding-scored** document-QA agent (FinanceBench) — Python program versus free-text-with-evidence, both failing without the ingested domain context, both chosen under the principle of Methodology: Domain-Aware Evaluation Without Self-Bias.

5.1. Use Case 1 — Scientific-Coding Agent on ScienceAgentBench

Given a scientific task and a dataset, the agent produces a self-contained Python program. We use **ScienceAgentBench** [22] — 102 tasks from 44 peer-reviewed papers across four disciplines, each **scored by execution** (the best agents solve roughly a third, so it is far from saturated). Its *dataset + optional expert knowledge* input maps directly onto Step 1, its execution scoring is self-bias-proof, and it is SciPy-native. Step 1 ingests the task and expert knowledge; Step 4 scores by executing the program, and failures (e.g. "loads the wrong columns") cluster back into the runbook.

5.2. Use Case 2 — Financial Document QA on FinanceBench

The contrasting agent answers open-ended questions over real 10-K filings. We use **FinanceBench** [23] — open-book QA with 10,231 evidence-linked questions, on which a strong retrieval-augmented model was wrong or refused on roughly four-fifths of a sampled set, so the task is genuinely hard. The agent **must** ground each answer in the supplied documents, and the safety dimension is intrinsic: a hallucinated figure or a wrong refusal on a regulated domain is the failure that matters. The pipeline is identical — Step 1 extracts the filing taxonomy and grounding constraints; Step 3 generates evidence-required and refusal-expected cases; Step 4 scores against cited evidence (cf. RAGTruth [24]). One pipeline thus spans a structured, execution-verified coder and an open-ended, grounding-verified QA agent.

6. Results

The experimental variable is the **domain context supplied to the golden-set generator (Step 3)**, on FinanceBench (Use Case 2), in three arms: **blind** (nothing), **documents only** (filing evidence text — the benchmark's own questions and answers are withheld, so no arm can paraphrase them), and **documents + runbook** (the same text plus Step 2's Domain Compliance Runbook). The generator runs in every arm under one identical system prompt — only the data sections of the user message differ — with k = 3 independent generations per arm; we report mean ± sd. The evaluation is of the **generated questions, not the agent's answers**: what a framework can *test for* is decided at generation time, so that is what we measure. Every number below is reproducible from the archived per-run prompts, the nine saved golden sets, and the analysis scripts. Generation ran on Claude Fable 5 via Claude Code [1], August 2026 (flat-rate subscription, default settings, no fixed seed; Limitations and Future Work). All metrics are rule-based and offline; no LLM judge is used.

6.1. *FinanceBench: what the generated golden set can test for*

Every arm generated **50** questions per run. Given the documents, the generator wrote questions that name specific issuers, periods, and line items with answers traceable to the filing text; given the runbook, it also tested the domain's compliance rules; given neither, it wrote issuer-specific but unanswerable questions. We quantify this three ways — a rule-based domain-accuracy metric, lexical relatedness, and unsupervised topic structure — and then show what each golden set is made of.

Domain accuracy of the questions. A rule-based scorer, applied identically to every arm, marks a generated question *domain-accurate* when it targets a real, specific financial fact verifiable against the raw filing data — the mean of a relevance axis (names a statement or line item) and a groundedness axis (the answer's figure is found in the raw data; else a named issuer; else company-less), with guardrail tests excluded as non-factual. Groundedness is a *documents* effect: **75%, 92%, and 92%** for blind, documents, and documents + runbook (mean of k = 3, sd ≤ 1 point; Figure 2). The runbook moves this metric by zero — by construction, since it contributes no figures.

Lexical corroboration. A second offline metric: each question's **domain relatedness** is its maximum TF-IDF cosine similarity to the closest real FinanceBench record (over all question, answer, and evidence text; scikit-learn `TfidfVectorizer` [25] with English stop words, unigrams, and sublinear TF). Documents shift questions toward real filings — median **0.193** blind versus **0.267** (documents) and **0.266** (documents + runbook), questions pooled over runs (Figure 2); the runbook leaves this metric unchanged too.

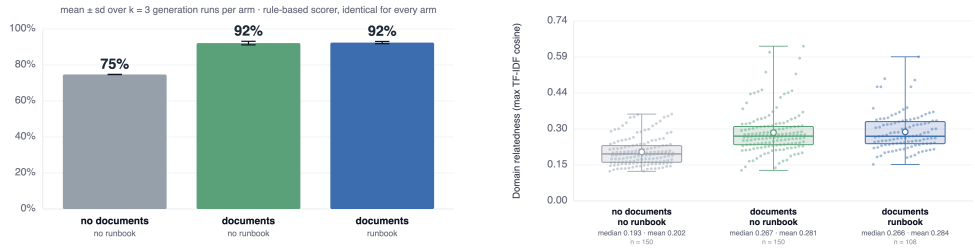


Figure 2. *Documents, not the runbook, drive question groundedness.* ***(a, left)*** *Domain accuracy (rule-based, identical scorer in every arm; mean ± sd over k = 3 runs): 75% blind, 92% with documents, 92% with documents + runbook.* ***(b, right)*** *Per-question domain relatedness (max TF-IDF cosine to the closest real filing), questions pooled over runs; the two documents arms overlap and both sit above blind.*

Topic structure and the compliance gap. Running NMF topic modeling [25] over the raw question text of each set (TF-IDF features, 1–2-grams, finance-boilerplate stop words; topic

assignments hand-verified by reading every question) — a view of what the generator actually produced — shows where the runbook acts (Figure 3, Figure 4). With the runbook, each 50-question set splits into **36 grounded capability questions** across seven finance topics plus **14 agent-compliance tests** (28% of the budget) spanning the four behavioral anchors: off-domain refusal, MNPI / confidential data, biased advice, and escalation (the fifth anchor, evidence-grounding, is enforced on every capability question).

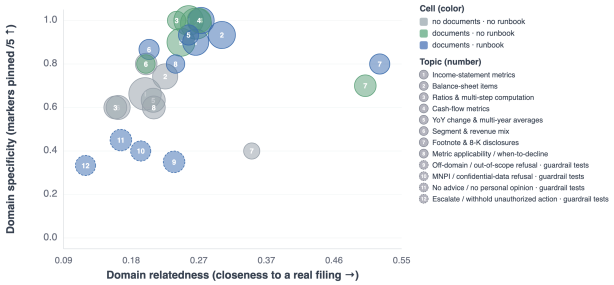


Figure 3. *NMF topics over the question text. x = mean domain relatedness (the TF-IDF metric of Figure 2); y = mean **domain specificity** — the fraction of five concreteness markers pinned down (named company, fiscal period, statement, line item, figure verifiable in a real filing). Bubble area = questions, colored by arm. Both documents arms'* capability *topics sit high and to the right; the documents + runbook arm's* guardrail tests *(dashed) sit low and left by design — naming an entity but using little filing vocabulary — and have no counterpart in the other arms; the blind arm's topics collapse into a less-grounded region.*

Without the runbook, the same generator under the same prompt produces **no guardrail tests at all (0/50 in every run of both other arms)** — confirmed by reading every question. This is *coverage by construction*: the shared prompt never requests refusal or safety cases; they are synthesized only from the runbook's anchors (Availability links every arm's prompt). The operative point is a *coverage* gap rather than a score swing: an evaluation built without the extracted anchors cannot test a single compliance rule, so an agent can breach all of them and the evaluation never knows — and the documents, which fix groundedness, do nothing to fix this. (A fourth, runbook-only condition in the repository's 2×2 replication reproduces the 14 guardrail tests with no documents at all.)

What each golden set is made of. Figure 4 splits every arm's 50-case budget by what a case can test. The runbook's effect on the *capability* cases is on the reference answer: a rule-based check counts how many of five provenance markers the runbook's Evidence anchor requires are present — source pointer, statement or line item, units and scale, fiscal period, GAAP / non-GAAP basis. Blind sets have no reference answers at all (50/50 unanswerable). Documents alone give the right figure with units but little else: 40 of 50 answers carry at most one marker, 10 carry two or three, none carry four or more. With the runbook, 36 of 36 answers carry four or five markers, and the remaining 14 cases are the guardrail tests across the other four anchors. Documents give the generator the figures; the runbook makes each answer carry its provenance and adds the guardrail tests — which is what an evaluation of a regulated-domain agent has to be able to test. Table 1 summarizes what each golden set can — and cannot — test for.

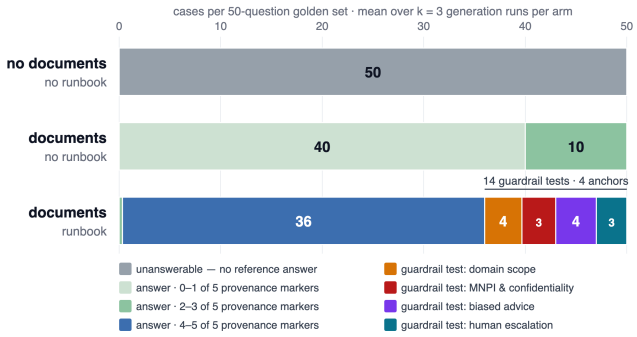


Figure 4. *Composition of each 50-case golden set (mean over k = 3 runs). Capability answers are tiered by how many of five provenance markers they carry (source, statement / line item, units, period, GAAP basis); guardrail tests — cases whose correct outcome is a refusal or a hand-off — are colored by the runbook anchor they test. The blind and documents-only sets contain* ***no guardrail tests*** *and no fully-sourced answer; the documents + runbook set is 36 fully-sourced answers plus 14 guardrail tests across four anchors.*

Table 1. *What each FinanceBench golden set can test for. The two baseline columns are the no-framework reality (not a measured run); the framework column is the measured documents + runbook golden set (run 1 of 3), whose 50 questions partition into 36 domain-capability questions and 14 guardrail tests; the documents-only arm reaches the same 36-question capability coverage with 0 guardrail tests.*

Capability	No golden set	Hand-authored	AI Eval Engine (documents + runbook)
A golden set exists to test against	—	yes	yes — 50 questions
Domain-capability / metric questions (income statement, balance sheet, cash flow, ratios, segments)	no	partial — common ones only	36 tests across 7 topic areas
of which: edge-case / decline (metric not meaningful, missing line item)	no	partial	2 tests (subset of the 36)
Guardrail tests (off-domain, MNPI, biased advice, escalation)	no	partial — only if remembered	14 tests (off-domain 4, MNPI 3, advice 4, escalation 3)
Systematic coverage of all 5 compliance anchors	no	ad hoc — no guarantee	yes — all 5 tested by construction
Versioned & reproducible	—	usually not	yes — content-addressed
Auto-regenerates when data / prompts change	—	no — goes stale	yes
Per-team authoring effort	none	high — from scratch, every team	~15-line YAML

6.2. Summary

The result is a *generation-time coverage* finding rather than an accuracy swing, and the intermediate arm separates the two ingredients. Documents make questions gradeable (75% to 92% groundedness) but leave 40 of 50 answers with at most one provenance marker

and add **zero** guardrail tests; the runbook adds nothing to groundedness but turns the set into 36 fully-sourced answers plus 14 guardrail tests spanning all four behavioral anchors. The added coverage exists *only because Step 1 ran*; a golden set built from the documents alone, scoring the very same agent, has no way to surface it.

7. Implementation Notes

The framework is Python (3.11+). Results exercises Steps 1–3; the Step-4 scorers and Step-5 metrics are implemented and unit-tested but not evaluated there. Only Step 1 needs a model call; Steps 2–5 run offline and deterministically, so the artifacts reproduce without an API key. Use Case 2 retrieves with Chroma [26].

8. Discussion

Reviewer, not author. A human stays in the loop at every generation step, so the framework optimizes for *acceptance latency*, not full autonomy. **Fixed anchors, discovered instances.** The five compliance anchors are a fixed taxonomy, guaranteeing coverage by construction, while their domain instantiations and the runbook's failure modes grow from observed behavior.

9. Limitations and Future Work

The current scope targets text- and code-producing agents; multimodal, long-horizon tool-using agents are future work. The runbook is a single JSON artifact that will need splitting at scale, and the Step 5 dashboard is read-only. Three experimental limits: the question metrics are rule-based (validated against an independent hand-labeling of 30 questions: MAE 0.09, Spearman ρ = 0.74); k = 3 unseeded generations per arm give only a coarse variance estimate; and one generator model family was used.

10. Conclusion

AI Eval Engine treats evaluation as a *pipeline to be generated*, not an artifact to be authored: from a pluggable domain context it generates the golden set, the eval script, and a living Domain Compliance Runbook — a domain-aware path to tracking agent behavior as a first-class OKR.

11. Availability

Source code is MIT-licensed at https://github.com/sbisen/ai-eval-engine; the nine golden sets, per-run archived prompts, scorer outputs, hand labels, and analysis scripts behind Results are under `results/ab_experiment/rerun_2026-08-28_3arm/`. The public API mirrors the paper's steps:

```
from ai_eval_engine import extract_domain_context, generate_golden_set
ctx = extract_domain_context("configs/financebench.yaml")  # Step 1 -> DomainContext
gs = generate_golden_set("configs/financebench.yaml", "out/context.json", runbook_path="out/
runbook.json")  # Step 3
```

All artifacts are plain JSON. From the committed FinanceBench run, a Step-2 runbook *domain fact* reads `{"group": "Definitions", "label": "Free cash flow", "detail": "Operating cash flow minus capex, from the cited cash-flow statement."}`; one Step-3 `GoldenCase` (abridged):

```
{"id": "wdc-001", "question_type": "metrics-generated",
 "input": "Using 3M's FY2018 consolidated statement of cash flows, what was 3M's
           capital expenditure (purchases of PP&E) in USD millions?",
 "expected": "$1,577 million. Source: 3M_2018_10K, Consolidated Statement of Cash Flows.",
 "probes_criteria": ["Citation & non-misleading disclosure"]}
```

12. Disclosures

Generative AI. Anthropic Claude (API and Claude Code) was both this work's *subject* — the framework calls it for extraction and generation — and a *writing aid*; the author reviewed and revised all outputs and takes full responsibility for the content. **Affiliation.** Independent open-source work, separate from the author's employer.

References

[1] Anthropic, "Claude." [Online]. Available: https://www.anthropic.com/claude

[2] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL): System Demonstrations*, 2024. doi: 10.48550/arXiv.2309.15217.

[3] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024. doi: 10.48550/arXiv.2311.09476.

[4] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, "G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. doi: 10.48550/arXiv.2303.16634.

[5] L. Zheng *et al.*, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023. doi: 10.48550/arXiv.2306.05685.

[6] P. Liang, R. Bommasani, T. Lee, and others, "Holistic Evaluation of Language Models," *Transactions on Machine Learning Research (TMLR)*, 2023, doi: 10.48550/arXiv.2211.09110.

[7] J. Ip and K. Vongthongsri, "DeepEval: The Open-Source LLM Evaluation Framework." [Online]. Available: https://github.com/confident-ai/deepeval

[8] Arize AI, "Arize Phoenix: Open-Source AI Observability and Evaluation." [Online]. Available: https://github.com/Arize-ai/phoenix

[9] Comet ML, "Opik: Open-Source LLM Evaluation and Observability." [Online]. Available: https://github.com/comet-ml/opik

[10] G. Guinet, B. Omidvar-Tehrani, A. Deoras, and L. Callot, "Automated Evaluation of Retrieval-Augmented Language Models with Task-Specific Exam Generation," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. doi: 10.48550/arXiv.2405.13622.

[11] S. Shankar, J. Zamfirescu-Pereira, B. Hartmann, A. G. Parameswaran, and I. Arawjo, "Who Validates the Validators? Aligning LLM-Assisted Evaluation of LLM Outputs with Human Preferences," in *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2024. doi: 10.48550/arXiv.2404.12272.

[12] S. Shankar *et al.*, "SPADE: Synthesizing Data Quality Assertions for Large Language Model Pipelines," *Proceedings of the VLDB Endowment*, 2024, doi: 10.48550/arXiv.2401.03038.

[13] S. Komoravolu and K. Mrini, "Agent-Testing Agent: A Meta-Agent for Automated Testing and Evaluation of Conversational AI Agents." arXiv preprint, 2025. doi: 10.48550/arXiv.2508.17393.

[14] W. Wang, Z. Ma, X. Wang, Y. Zhang, P. Liu, and M. Chen, "TestAgent: Automatic Benchmarking and Exploratory Interaction for Evaluating LLMs in Vertical Domains." arXiv preprint, 2024. doi: 10.48550/arXiv.2410.11507.

[15] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "\(\tau\)-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains". arXiv preprint, 2024. doi: 10.48550/arXiv.2406.12045.

[16] A. Yehudai *et al.*, "Survey on Evaluation of LLM-based Agents." arXiv preprint, 2025. doi: 10.48550/arXiv.2503.16416.

[17] M. Mohammadi, Y. Li, J. Lo, and W. Yip, "Evaluation and Benchmarking of LLM Agents: A Survey," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2025. doi: 10.48550/arXiv.2507.21504.

[18] Z. Hui, Y. R. Dong, E. Shareghi, and N. Collier, "TRIDENT: Benchmarking LLM Safety in Finance, Medicine, and Law." arXiv preprint, 2025. doi: 10.48550/arXiv.2507.21134.

[19] B. Wang *et al.*, "DecodingTrust: A Comprehensive Assessment of Trustworthiness in GPT Models," in *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023. doi: 10.48550/arXiv.2306.11698.

[20] Q. Zhang *et al.*, "Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models." arXiv preprint, 2026. doi: 10.48550/arXiv.2510.04618.

[21] P. Yuan *et al.*, "Silencer: From Discovery to Mitigation of Self-Bias in LLM-as-Benchmark-Generator." arXiv preprint, 2025. doi: 10.48550/arXiv.2505.20738.

[22] Z. Chen *et al.*, "ScienceAgentBench: Toward Rigorous Assessment of Language Agents for Data-Driven Scientific Discovery," in *International Conference on Learning Representations (ICLR)*, 2025. doi: 10.48550/arXiv.2410.05080.

[23] P. Islam, A. Kannappan, D. Kiela, R. Qian, N. Scherrer, and B. Vidgen, "FinanceBench: A New Benchmark for Financial Question Answering." arXiv preprint, 2023. doi: 10.48550/arXiv.2311.11944.

[24] C. Niu *et al.*, "RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models." arXiv preprint, 2024. doi: 10.48550/arXiv.2401.00396.

[25] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[26] Chroma, "Chroma: The Open-Source Embedding Database." [Online]. Available: https://github.com/chroma-core/chroma